Become a Solution Architect

Enhance your Technical and Behavioural Skills

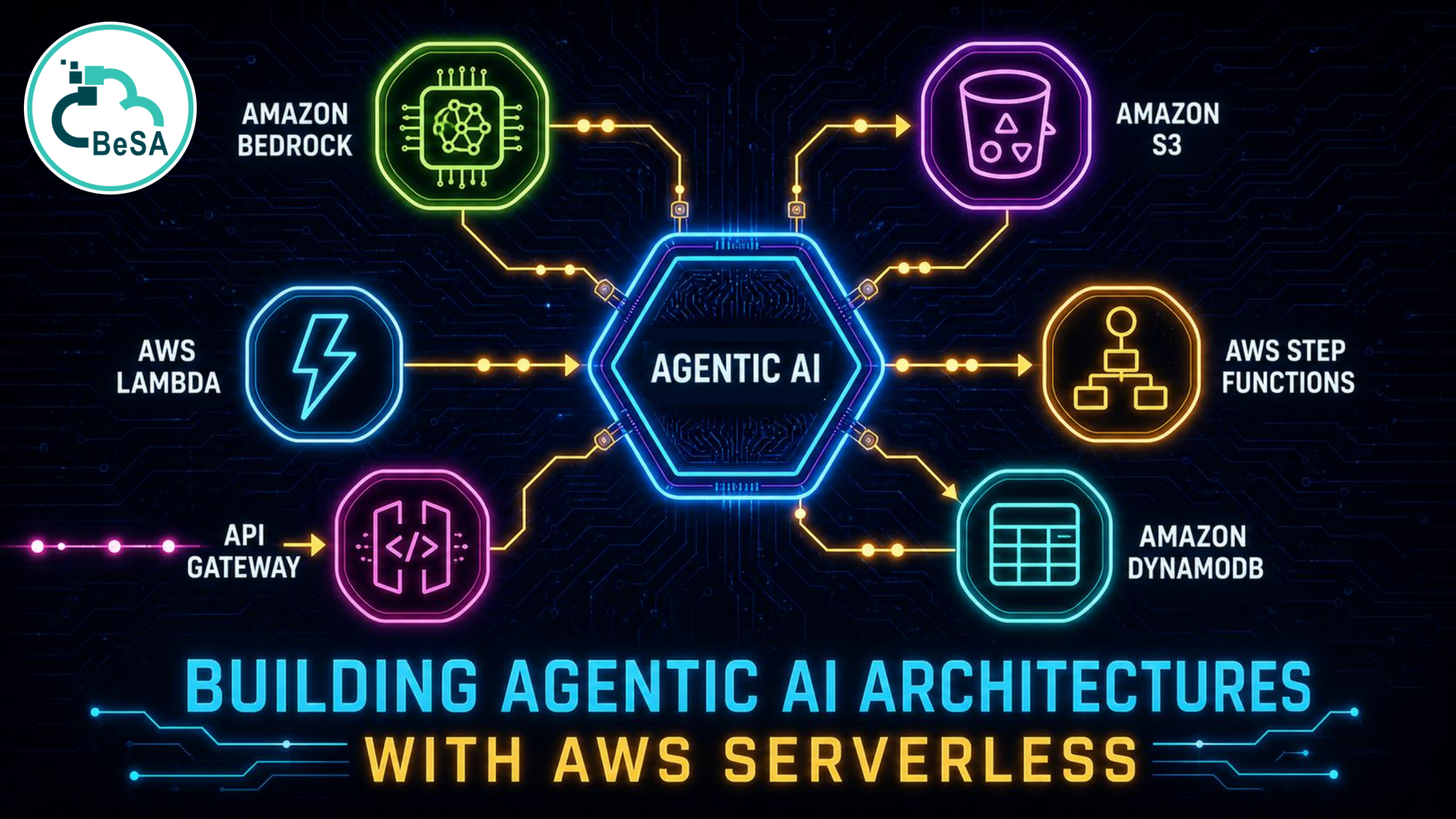
Did you post about your Week 1 Badge on Social Media?

Week 2 - Agenda

Topic	Duration	Presenter
Technical Topic Building Agentic AI architectures with AWS Serverless	50 Mins	Ashish & Krishna
Behavioural Topic Why Facilitation is the SA Superpower	30 Mins	Jeff & Nisha
Workshop Walkthrough	30 Mins	Parna

Pre Launch Q&A Session

Meet the team


BeSA
AMAZON BEDROCK
AMAZON S3
AWS LAMBDA
AGENTIC AI
AWS STEP FUNCTIONS
API GATEWAY
AMAZON DYNAMODB
BUILDING AGENTIC AI ARCHITECTURES
WITH AWS SERVERLESS

Analogy - Serverless

OWNING A CAR

(Traditional Servers)

You are responsible for:

- Buying the car
- Maintenance
- Insurance
- Keeping it available
- Capacity whether you use it or not

TAKING A TAXI

(Serverless)

You simply:

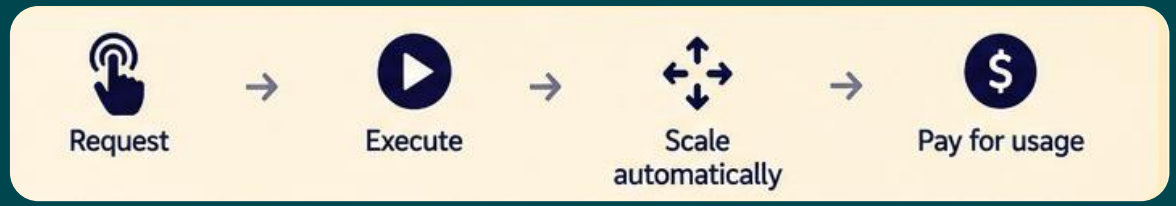
- Request it when needed
- Use it
- Pay for the journey
- Leave the operation to the provider

Some More Services In Real-life

What Is Serverless?

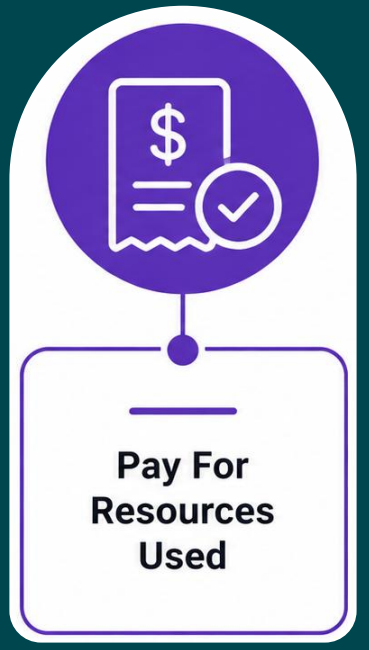
- Serverless ≠ No Servers

- Serverless = Servers are abstracted away from the application developer



The Business Case Of Serverless AI

- AI workloads, particularly inference, are often intermittent, unpredictable and bursty. In traditional architectures, this leads to overprovisioned infrastructure, increased costs, and complexity in scaling.
- In contrast, serverless architectures offer significant advantages.



AWS Services Powering Serverless AI (Partial List)

AWS Service	What it offers?	Why ideal for Agentic AI?
AWS Lambda	Event-driven compute	Execute agent tools and actions
SageMaker Serverless Inference	On-demand ML inference	Run predictions without dedicated infrastructure
Amazon Bedrock	Access to foundation models	Provide reasoning and intelligence
Amazon Bedrock AgentCore	Runtime, memory, identity & tools	Build and operate production agents
Amazon EventBridge	Event routing	Trigger and connect agents asynchronously
AWS Step Functions	Workflow orchestration	Coordinate deterministic, multi-step workflows
AWS IoT Greengrass	Edge runtime	Run AI logic closer to devices
Lambda@Edge	Edge compute	Execute lightweight logic near users

Core Principles of Serverless AI on AWS

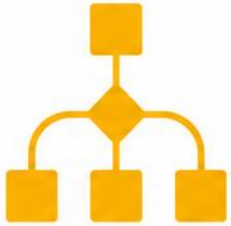
Core Principles of Serverless AI

- Serverless AI combines intelligence on demand with infrastructure on demand.
- Building serverless AI requires answering five fundamental architecture questions:

1

What starts the system?

2

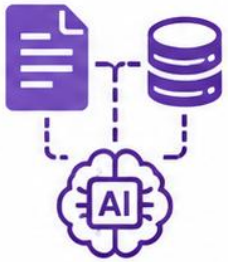


Who coordinates the work?

3

Where does intelligence execute?

4



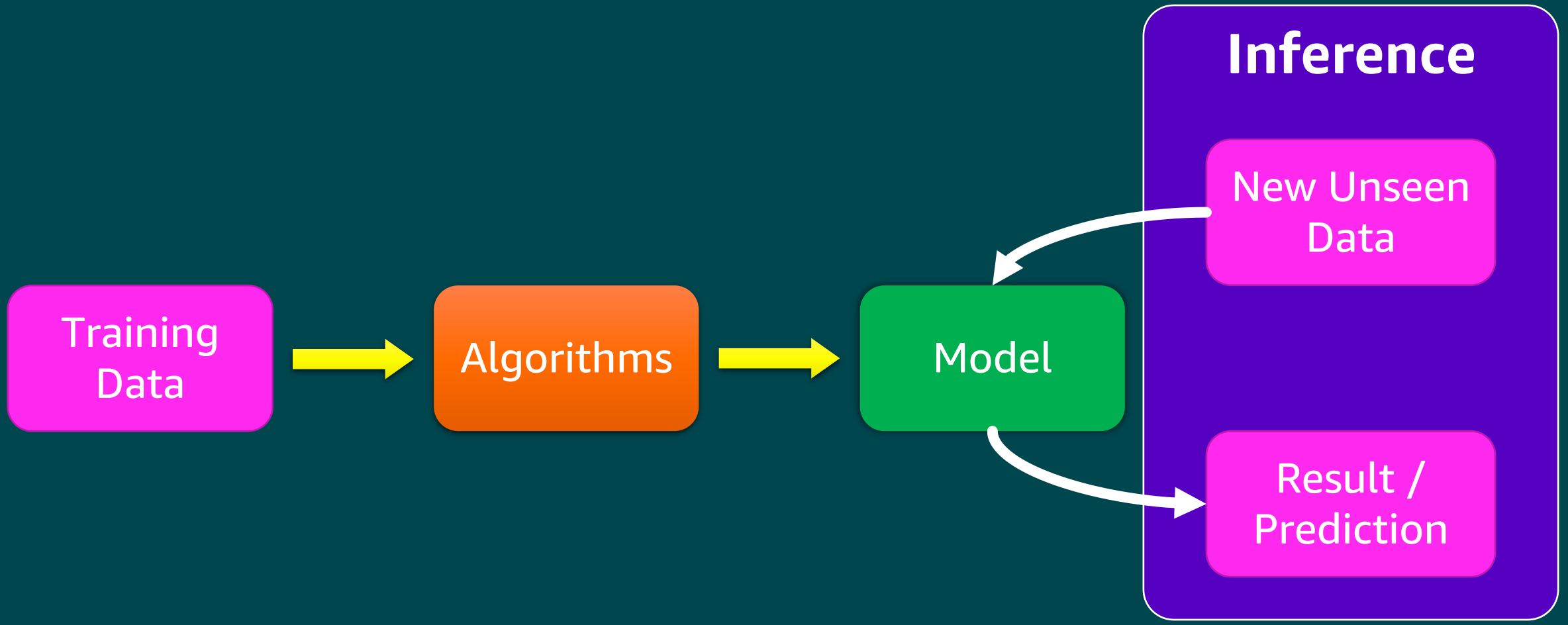
Where does AI get trusted context?

5

Where should inference happen?

Inferences

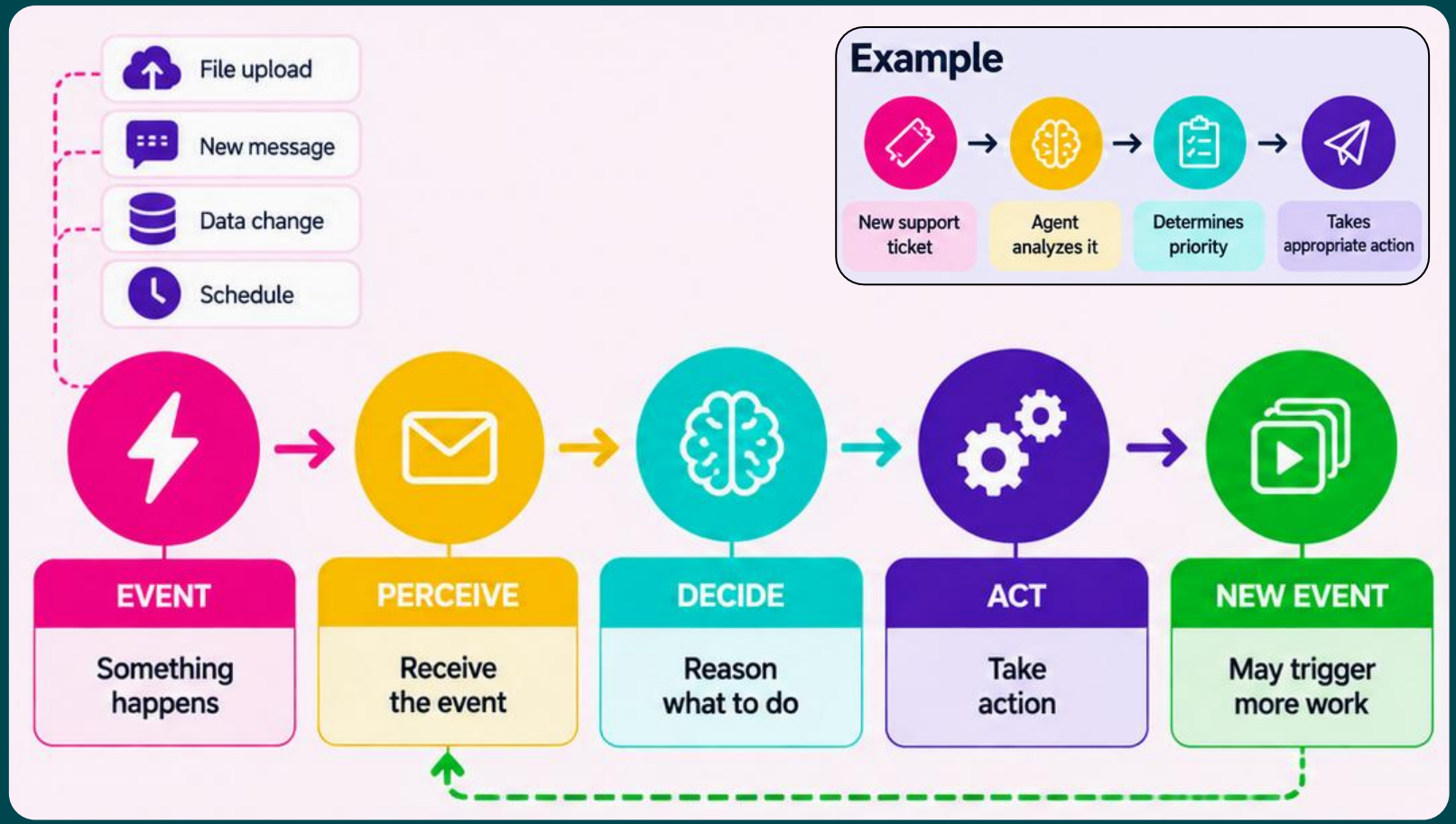
- Inference is an ML model in action. Inference is the process of running unseen data through a trained AI model to make a prediction or solve a task.



Event-Drive Architecture: The Backbone of Serverless AI

- Events allow AI systems to perceive changes and react without continuously running infrastructure.

Something happens → system wakes up → intelligence decides → action is taken.



Useful AWS Services

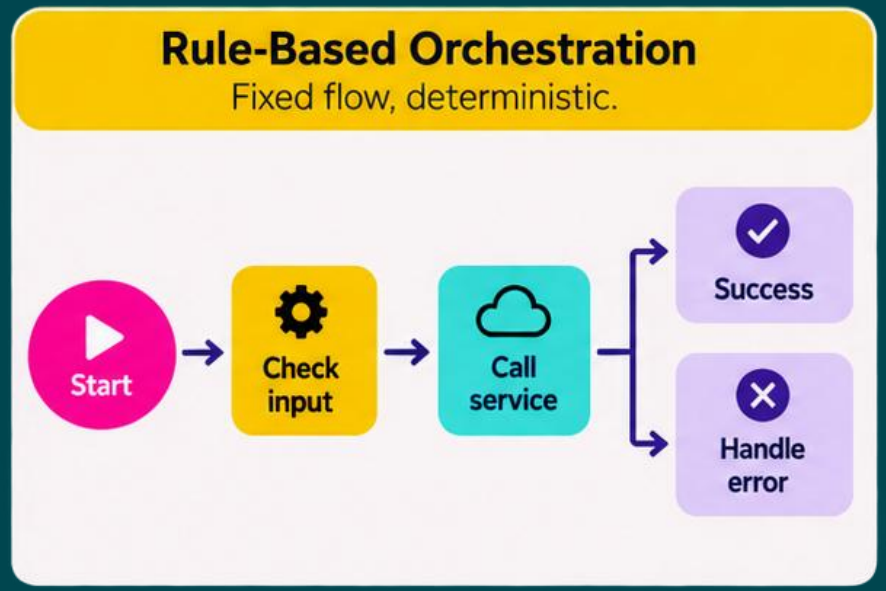
Amazon EventBridge

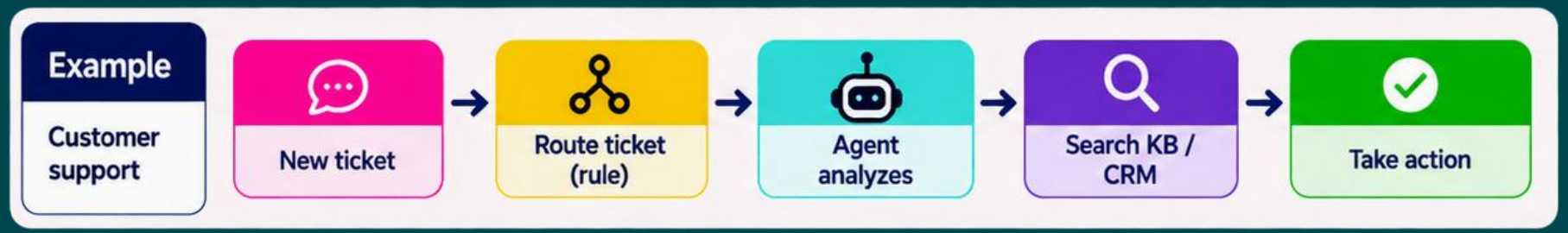
Amazon Simple Queue Service (Amazon SQS)

AWS Lambda

Orchestration: From Rule-Based to AI-Native

- Orchestration decided what happens next and coordinates services, tools and agents.





Useful AWS Services

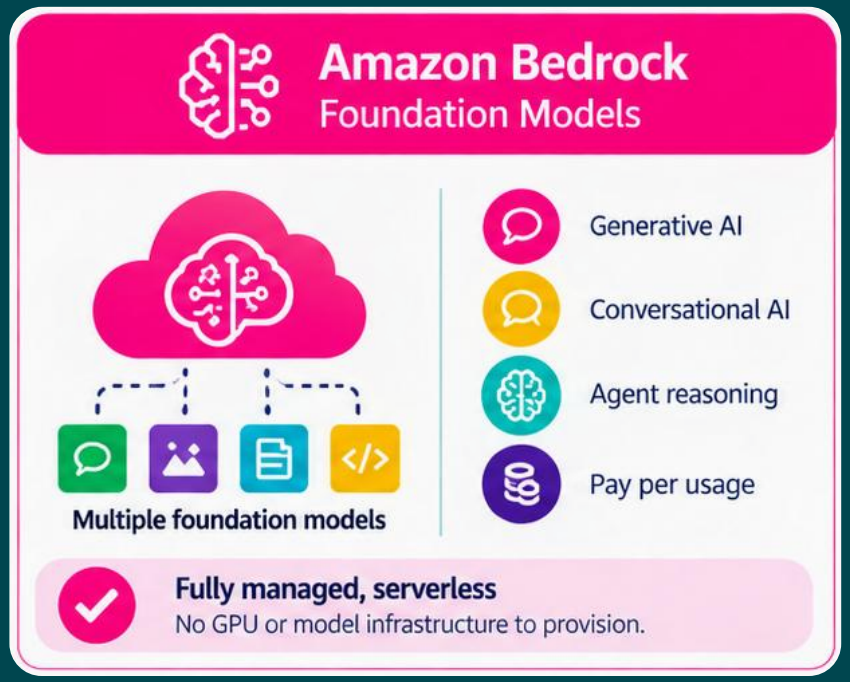
AWS Step Functions

Amazon Bedrock AgentCore

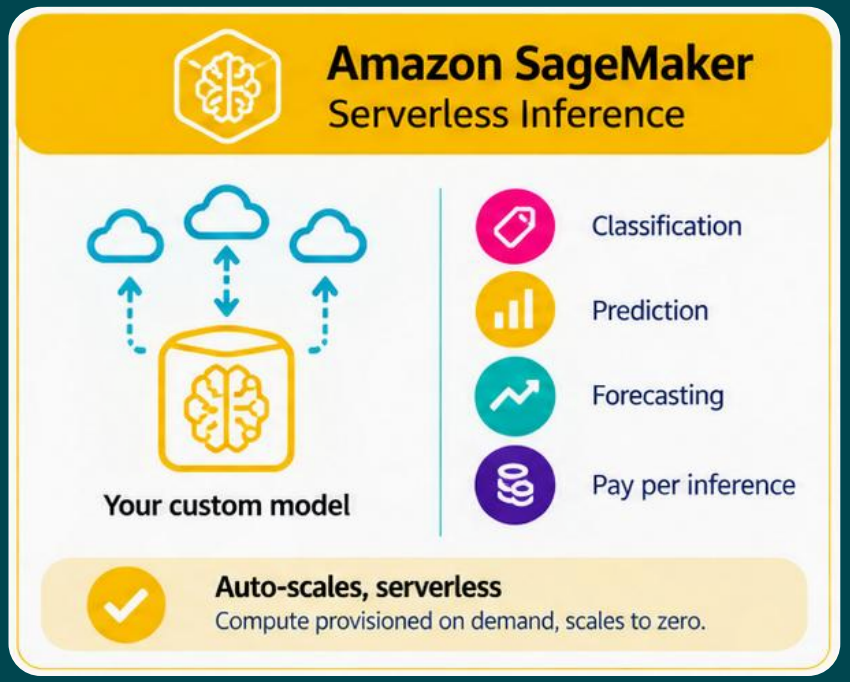
Model Execution Strategies for AI Workloads

- Two serverless paths:

Need popular foundation models



Have your own model



Useful AWS Services

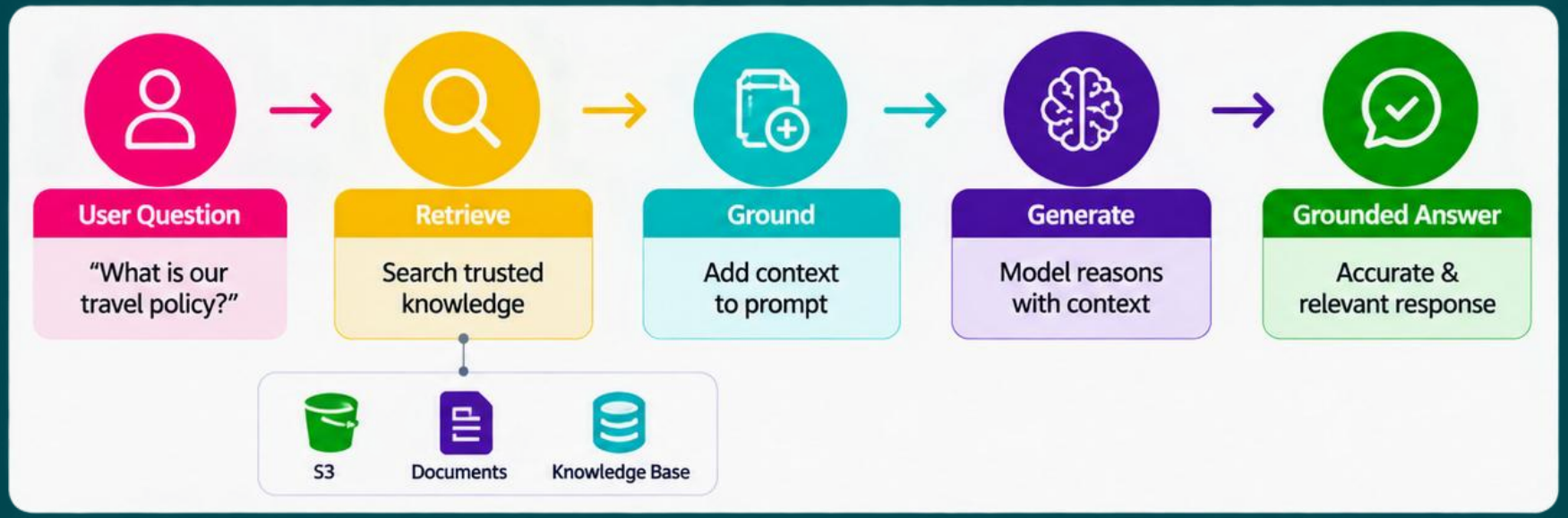
Amazon Bedrock

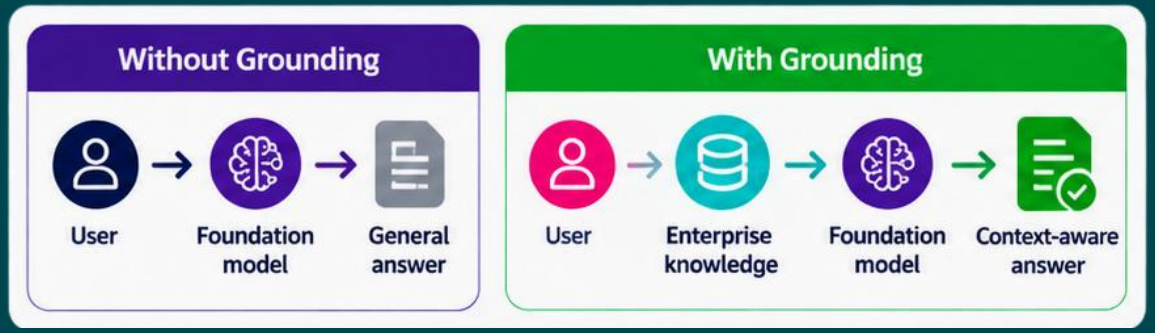
Amazon SageMaker AI

Grounding & RAG: Connecting AI to Trusted Knowledge

- Foundation models know a lot – but they don’t automatically know your latest or private business data.

RAG turns general-purpose intelligence into domain-aware intelligence.





Useful AWS Services

Amazon Bedrock Knowledge Bases

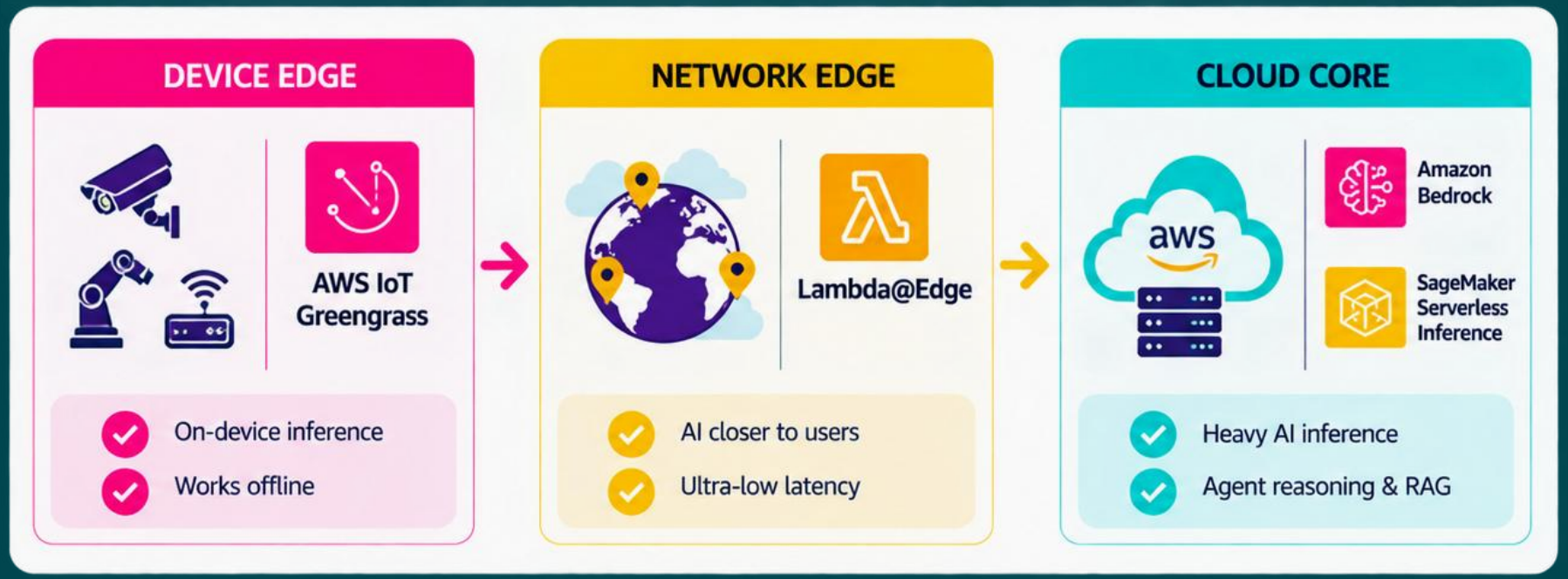
Amazon Simple Storage Service (Amazon S3)

Amazon OpenSearch Service Serverless

Where Should Inference Happen?

- Run AI at the right place based on latency, connectivity, model size and where the data is generated.

Not every AI decision needs a round trip to the cloud.



Useful AWS Services

AWS IoT Greengrass

AWS Lambda@Edge

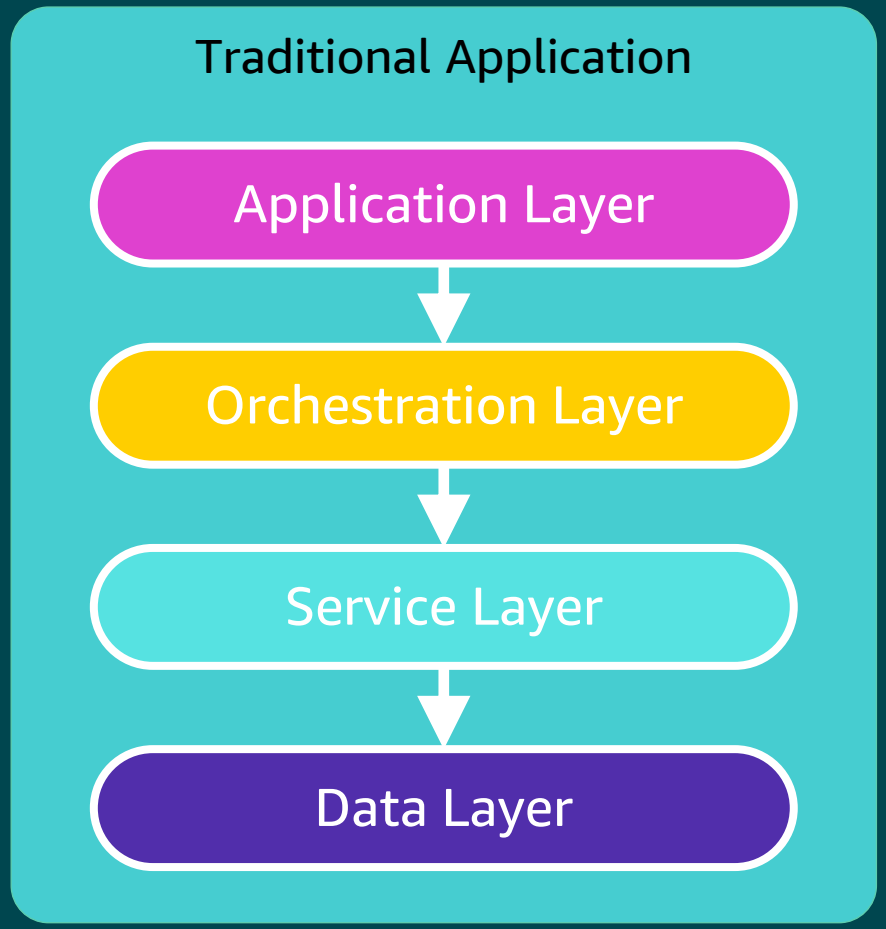
Amazon Bedrock

Amazon SageMaker Serverless Inference

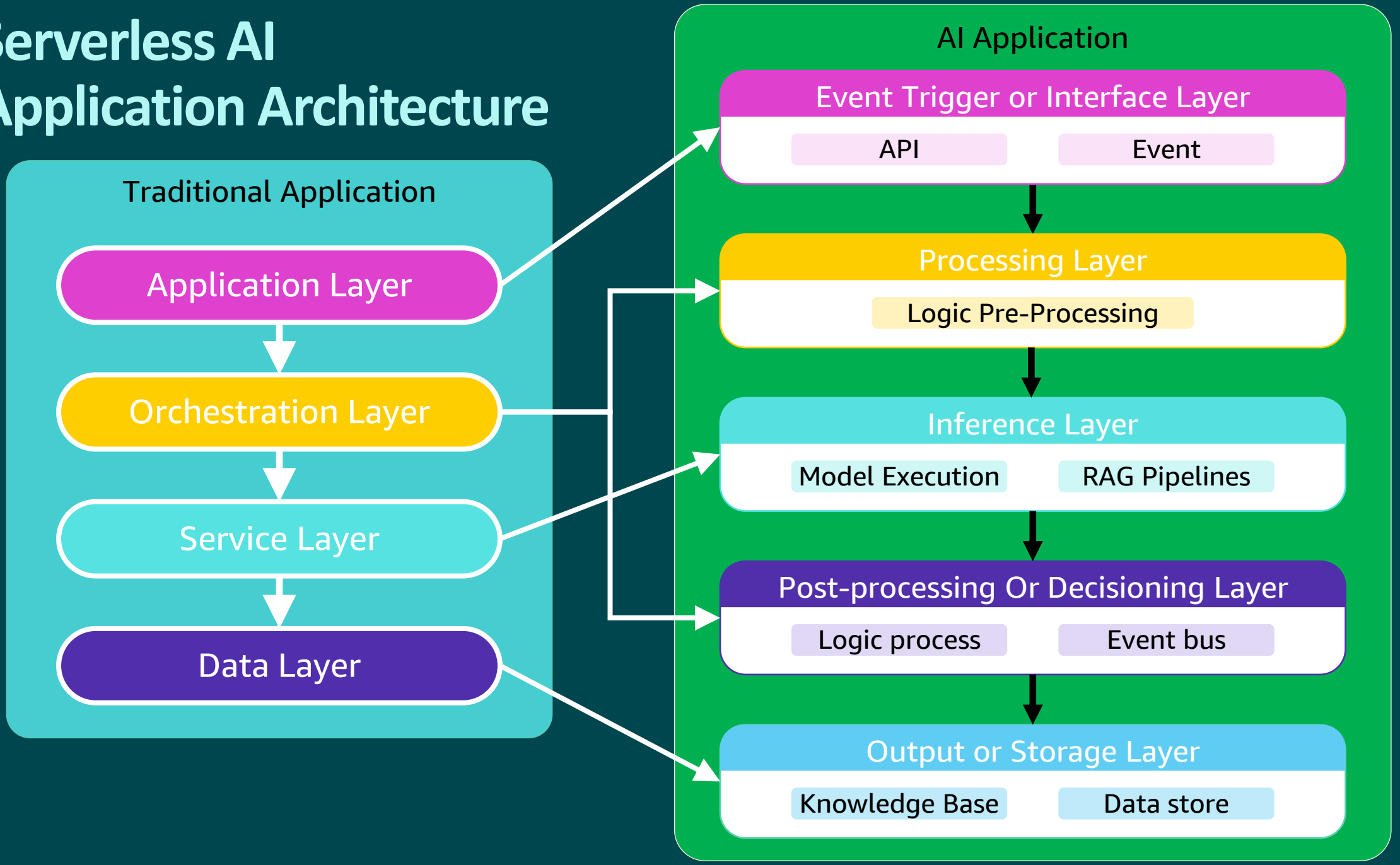
Application Architecture

Application Architecture

- In a traditional Event Driven Architecture (EDA), the system is structured into four logical layers

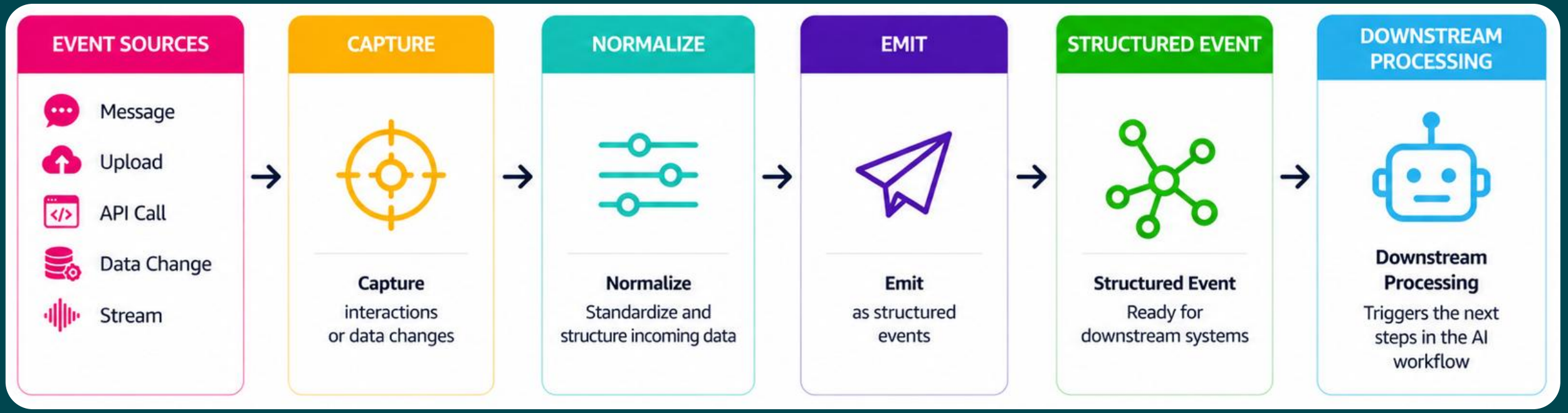


Serverless AI Application Architecture



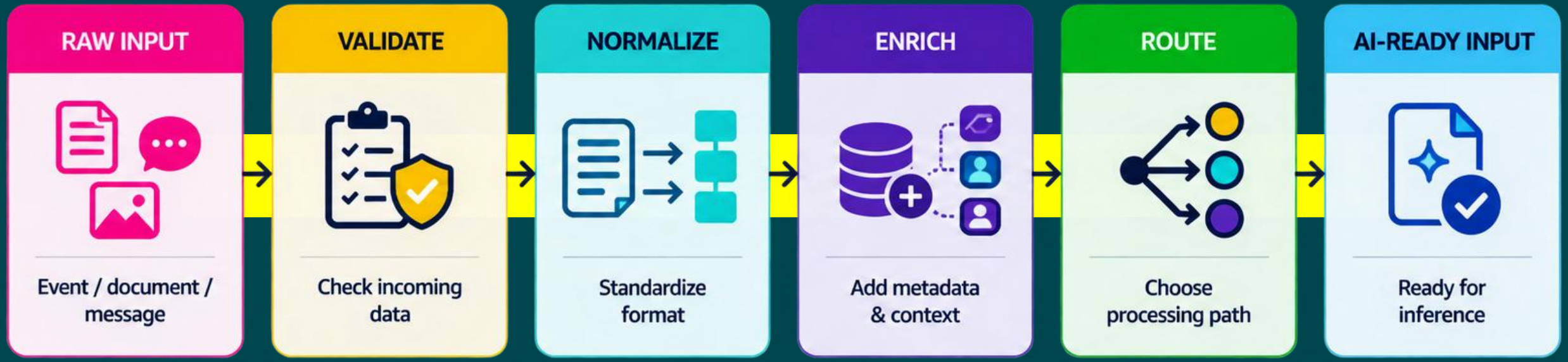
1. Event Trigger / Interface Layer

- Captures what happens and turns into an event the AI system can process.



2. Processing Layer

- Prepares and enriches incoming data before it reached the AI model.

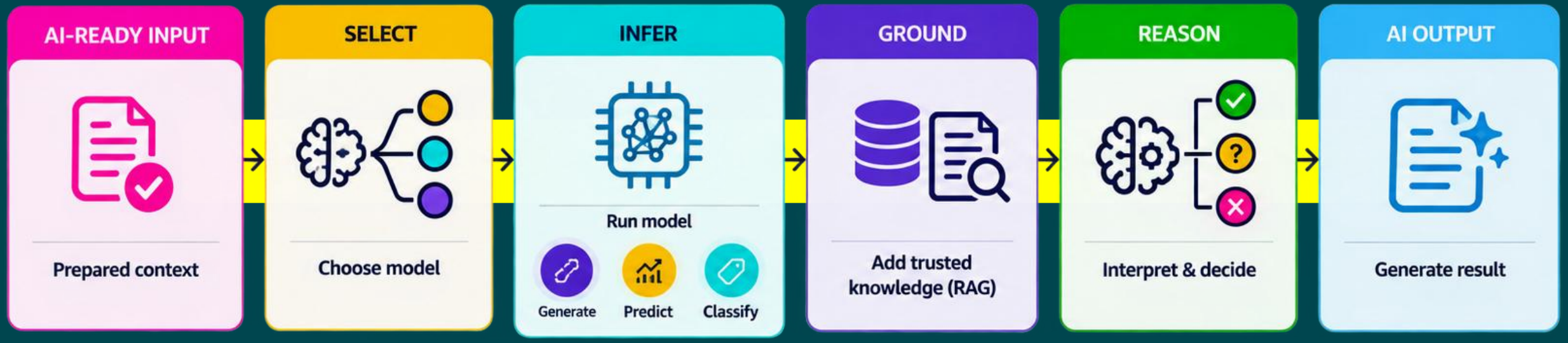


EXAMPLE

Raw documents are cleaned and enriched before being sent to the AI model.

3. Inference Layer

- Applies AI intelligence to generate, predict, classify or reason.



EXAMPLE

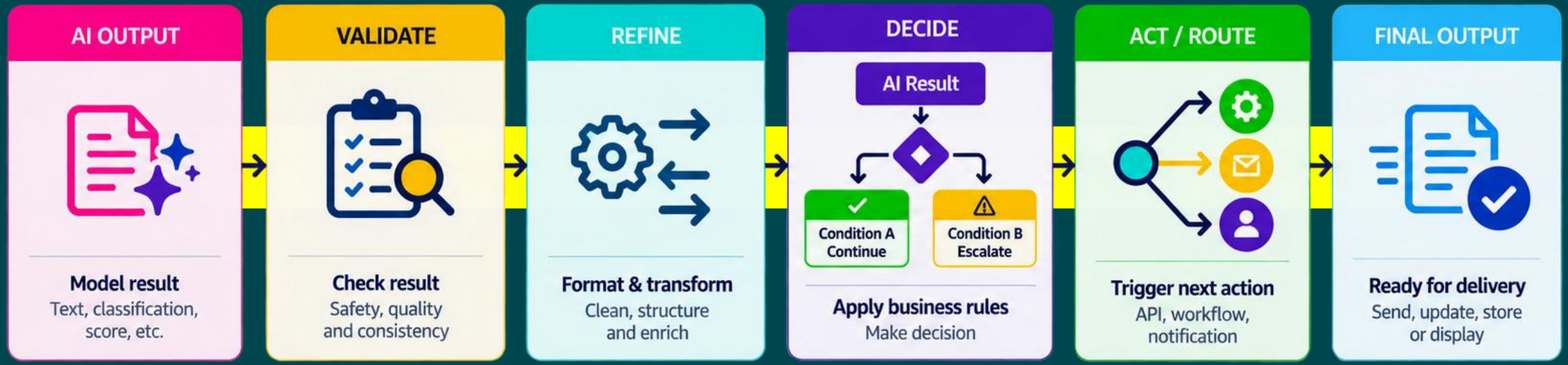
The model combines the prepared input with trusted knowledge to generate a relevant result.

Also used for:

SageMaker Serverless
Custom ML models

4. Post-Processing / Decisioning Layer

- Turns AI output into a validated response, business decision or action.

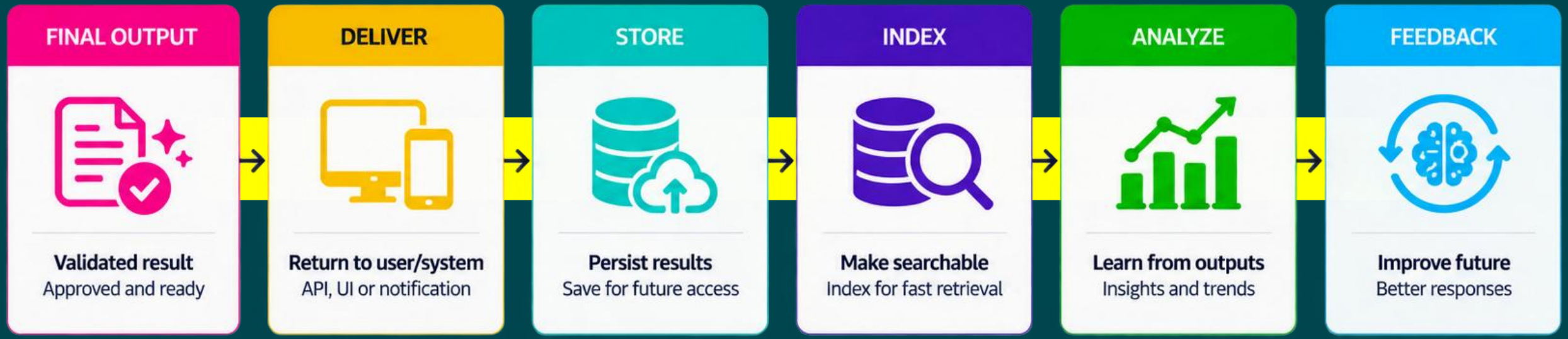


EXAMPLE

AI generates the insight; application logic determines the appropriate action.

5. Output / Storage Layer

- Delivers AI results and preserves them for future use, analysis and improvement



EXAMPLE

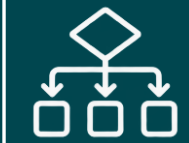
The result is delivered now and preserved to create value later.

Use Cases

Retail Customer Service Agent

Use Case	Layers	Architecture
 Customer submits a support request "I want to return this order"	**1. Event Trigger / Interface**	Use Amazon API Gateway, chatbot UI, or support portal to trigger agent interaction through Amazon Bedrock
 Request is prepared and passed to the agent	**2. Processing**	Implement Lambda to format input, apply security context (for example, user roles or entitlements), and enrich metadata
 Agent understands intent, checks order status and retrieves return policy	**3. Inference**	Use Amazon Bedrock agent to receive the prompt, invoke Lambda tools (for example, getOrderStatus), perform grounding through a knowledge base (e.g. return policy), and assemble a final response
 Return eligibility is evaluated and return is initiated when eligible	**4. Post-Processing / Decisioning**	Use Lambda to inspect agent output (for example, escalate if "order lost" and notify support team)
 Response is returned to the customer; interaction can be persisted	**5. Output / Storage**	Returns agent response to UI or logs it to Amazon S3 or Amazon OpenSearch Service for audit, training, or analytics

Legal Document Ingestion And Summarization

Use Case	Layers	Architecture
 A legal contract PDF is uploaded for analysis	**1. Event Trigger / Interface**	Amazon S3 → EventBridge → AWS Step Functions starts the workflow.
 Document metadata is prepared and input is enriched	**2. Processing**	AWS Lambda prepares metadata, detects language, and classifies file type.
 Text is extracted, classified, summarized, and risk is explained	**3. Inference**	Amazon Textract → SageMaker model → Amazon Bedrock, orchestrated by Step Functions.
 Risk and document type determine the next action	**4. Post-Processing / Decisioning**	AWS Lambda applies routing logic such as review, legal escalation, or auto-process.
 Results are stored, indexed, and alerts are emitted	**5. Output / Storage**	Store outputs in Amazon S3, index in Amazon OpenSearch Service, and emit alerts/audit events through Amazon SNS or EventBridge.

Factory Equipment Monitoring: Real-Time Inference at the Edge

Use Case	Layers	Architecture
 Equipment sensors continuously generate vibration and device-state data	**1. Event Trigger / Interface**	Edge device events such as sensor readings and device-state changes trigger processing
 Sensor data is cleaned and prepared locally	**2. Processing**	Local Lambda function on AWS IoT Greengrass formats input, extracts metadata, and filters noise
 Equipment data is analyzed locally to detect abnormal behavior	**3. Inference**	ML model such as PyTorch or ONNX runs locally as an AWS IoT Greengrass component
 Detected anomalies trigger an immediate local response	**4. Post-Processing / Decisioning**	AWS IoT Greengrass publishes anomaly events through MQTT or AWS IoT Device Shadows
 Operator receives alerts while summarized data is synchronized to the cloud	**5. Output / Storage**	Results can synchronize through AWS IoT Core, Amazon S3, or Amazon EventBridge; only relevant summary data needs to reach the cloud

Design Considerations Across Layers

Design Considerations Across Layers

Resilience
- Each layer should fail and retry independently (for example, dead-letter queues (DLQs) on Lambda).

Observability
- Emit structured logs, traces, and metrics from each stage to Amazon CloudWatch to detect behavioral drift.

Security
- Use AWS IAM role separation and AWS Key Management Service for data encryption across layers.

Cost optimization
- Use asynchronous execution where possible and choose right–sized models.

Extensibility
- Modular design allows services to be replaced or upgraded independently.

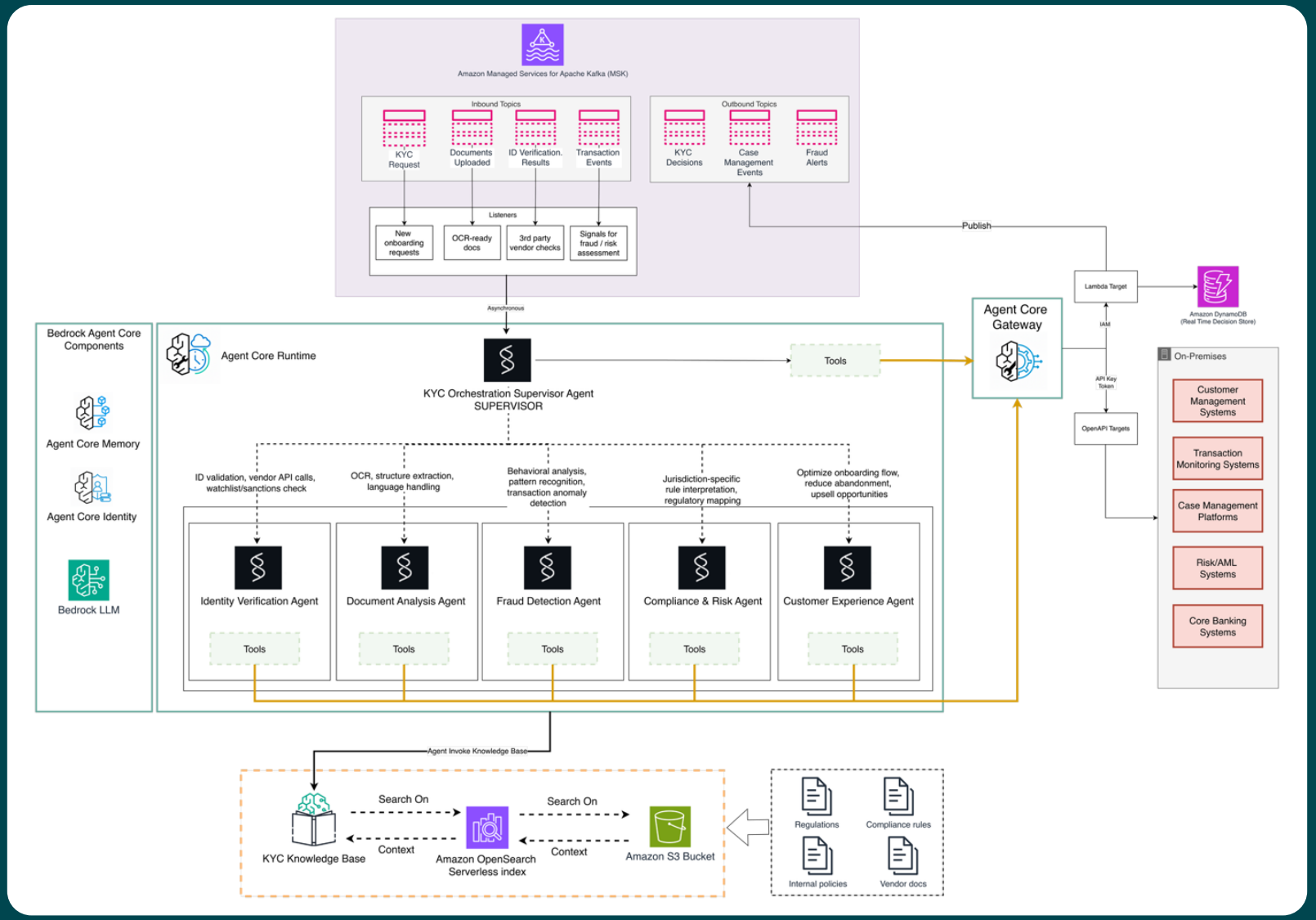
Implementation Strategies

Implementation Strategies For Serverless AI

Category	AWS Services / Techniques / Tools
Infrastructure as Code (IaC)	AWS CDK, AWS CloudFormation, AWS SAM; Terraform; Git/version control; environment-specific stacks; drift detection; schema validation & linting
Prompt, Agent & Model Lifecycle Management	Amazon Bedrock Prompt Management, Amazon Bedrock Knowledge Bases, Amazon Bedrock AgentCore (Runtime, Memory, Gateway), AWS CDK, CloudFormation, CloudWatch Logs, AWS X-Ray; Git; prompt versioning; golden test cases; confidence thresholds; shadow deployments; model version tracking
Testing & Validation	AWS CodePipeline, AWS CodeBuild, Amazon Bedrock AgentCore Harness, Amazon Bedrock AgentCore Evaluations, API Gateway models, CloudWatch, AWS X-Ray; pytest/unittest; Promptfoo; JSON Schema/Pydantic; golden prompt tests; agent tool tests; workflow integration tests; human-in-the-loop evaluation
Observability & Monitoring	Amazon CloudWatch Logs & Metrics, AWS X-Ray, Bedrock agent trace/model invocation logging, AWS CloudTrail, Amazon OpenSearch Service, CloudWatch Synthetics; structured logging; custom metrics; trace IDs; alarms
Security & Governance	IAM, AWS KMS, AWS CloudTrail, AWS Config, Amazon Macie, Amazon Bedrock Guardrails, Amazon Bedrock AgentCore Identity, Amazon Bedrock AgentCore Policy, CloudWatch Logs Insights; least privilege; scoped agent tools; prompt validation; encryption; output filtering; audit logging
CI/CD & Automation	AWS CodePipeline, AWS CodeBuild, AWS CDK, CloudFormation, AWS SAM, Amazon S3, AWS Lambda, Amazon Bedrock API/CLI, CloudWatch Synthetics, EventBridge, AgentCore; automated prompt regression tests; deployment approvals; rollback/versioning
Cost Optimization	Amazon Bedrock model tiers, Amazon Bedrock prompt caching, CloudWatch, AWS Budgets, Cost Explorer, DynamoDB caching, EventBridge + SQS + Lambda; tiered model routing; token limits; scoped RAG; batching; async processing; caching; confidence-based escalation; cost allocation tags

Modernizing KYC with AWS serverless solutions and agentic AI

- High-level Agentic Architecture for real-time KYC



Why Facilitation is the SA Superpower

Hands-on Workshop

Walkthrough

Next Steps: Your Turn to Act

1

Complete the Knowledge Check

2

Practice the Hands-on Workshop

3

Be active and Support Your Peers

4

Share Your Learning on Social Media